S2 E14

BK

Season 2: Retrieval and RAG

Tracing a wrong answer.

When the system gets an answer wrong, which stage caused it? Every wrong answer, walked back in code to the first stage that lost the right passage. No new model calls.

Brendan Kelly

Applied AI

Why it matters

“The bot got it wrong” does not say where

1	2	3	4	5	6	7	8	9	10	11
Load	Chunk	Embed	Rewrite	Search	Filter	Rerank	Prompt	Answer	Cite	Score

A REAL TEST QUESTION, ANSWERED BY THE FINAL SYSTEM

What is part KX-3673 for? Answer: the removable filter inside the water collection tank

The system replied: "I could not find what part KX-3673 is for in the passages." The library holds the answer. Eleven stages ran. Which one let it down?

Guessing is costly	A bigger model, a new prompt, a different search: each costs a test run, and the wrong fix changes nothing.

This episode

The final system still gets 46 of 400 answerable questions wrong. Trace each one to the stage that caused it.

Mental model

The pipeline is a relay, and every stage keeps a log

Each stage receives passages from the one before and hands some on. The log records what it received and what it passed.

5. Search
the right passage ranks 39th of 50

→

7. Rerank
reads only the top 20: 39th never arrives

→

8. Prompt
the top 5, without the right passage

The idea

Like a dropped baton: find the first runner who did not pass it on. Everything after that point never had a chance.

Why it is possible

The test questions come with labels marking which passage holds each answer. Code can follow that passage through the log.

So the pattern is

Log what every stage received and passed on. Without the log, a wrong answer cannot be traced.

Mental model

Seven places the right passage can be lost

Each wrong answer is filed under the first that happened.



Live, every question

The whole diagnosis is one function

It reads one question’s log and returns the first stage that lost the right passage. No model call.

src/lab/rag/score.py — the slide function

```
def first_lost_stage(trace, k, verdict):
    """The first stage that lost the right passage, or
    None if the answer came out right."""
    if trace["cut"]:
        return "chunk"
    if trace["searched"] is None:
        return "search"
    if trace["filtered"] is None:
        return "filter"
    if trace["reranked"] is None or trace["reranked"] > k:
        return "rerank"
    if verdict.correct:
        return None
    return "prompt" if verdict.declined else "answer"
```

Shortened. For the final system, "rerank" is split in two by the passage’s place after search: outside the top 20, or inside it.

Worked example

One wrong answer, traced stage by stage

THE QUESTION FROM SLIDE 2

What is part KX-3673 for?

Chunk	The passage with the answer is whole, not split.	kept
Search	It ranks 39th of 50: found, but low.	39th
Filter	Public and current, so it stays: still 39th.	kept
Rerank	The reranker reads only the top 20. 39th never reaches it.	lost
Answer	“I could not find what part KX-3673 is for in the passages.”	wrong

The verdict: outside the shortlist

The model did nothing wrong: it was never given the answer. A better answer model would not fix this. Better search would.

The test: trace every wrong answer in three systems

No new runs. Three systems from this season, each already run on all 520 test questions.

1. **E2: the first, naive system**
   Fixed-size chunks, plain search, no filter, no reranker. Judged by an earlier judge model.

2. **E4: the same system** the like-for-like start
   Rerun under the season's final judge and answer model, so it compares fairly with the final system.

3. **The final system**
   Everything that joined this season: section chunks, the filter, the reranker, the grounded prompt.

A wrong answer	An answerable question the judge did not mark right. Each one is filed under the first stage that lost its passage.

What was expected, written before the analysis

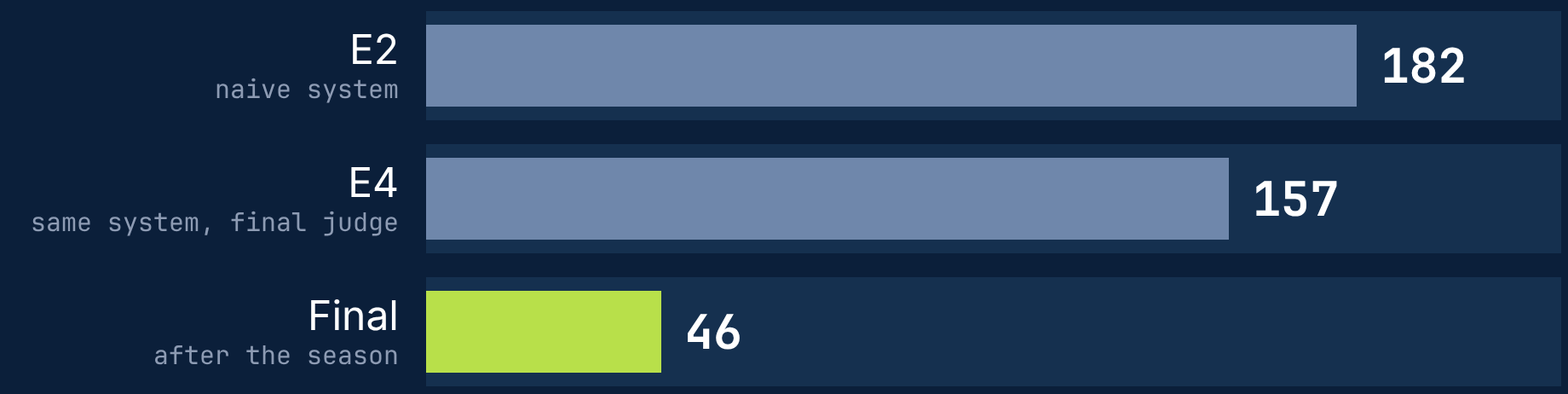
Three predictions about the final system, each with a pass mark set in advance.

Before the reranker	Of the answers lost at ranking, at least half had the passage outside the reranker’s top 20.
Two kinds of question	Follow-ups and part codes make up at least 60% of the wrong answers.
Detail, not topic	When it answers a question it should decline, the library usually covers the topic but not the detail.

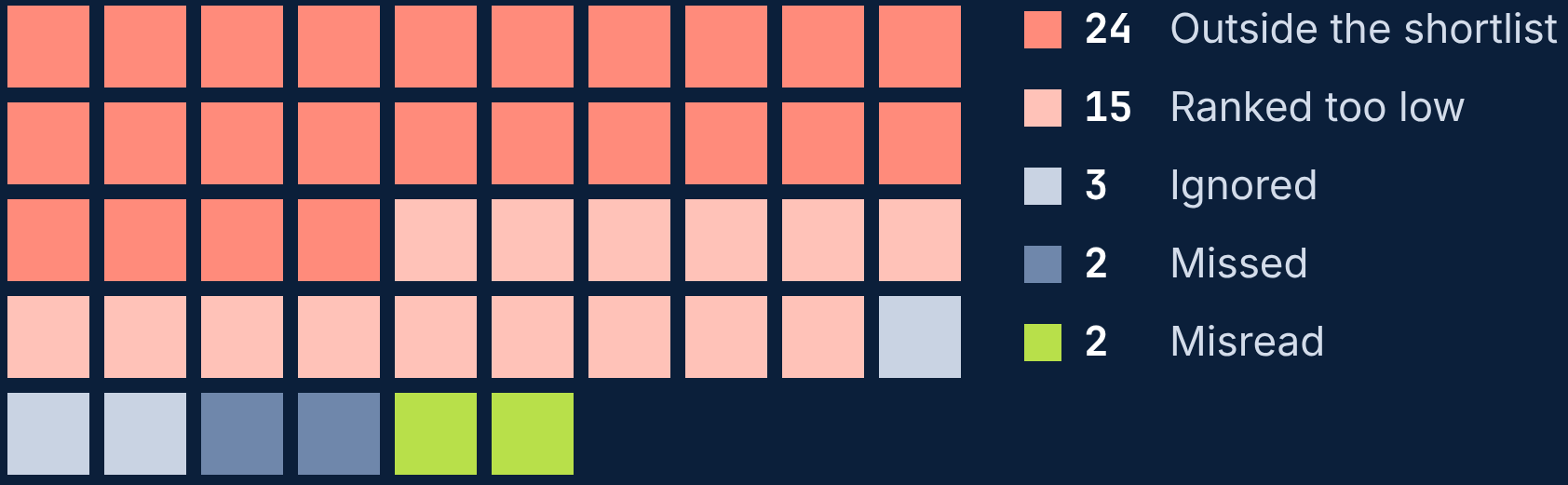
Finding

Wrong answers fell from 157 to 46

Wrong answers out of 400 answerable questions.



THE 46 THAT REMAIN, BY THE FIRST STAGE THAT LOST THE PASSAGE



What is left

39 of the 46, the two shades of coral, are ranking losses. That is where the next fix belongs.

Compare from E4, not E2: E2 used a different answer model and judge.

Finding

What each change to the system fixed

From E4 to the final system, stage by stage, with the episodes that changed that stage.

	E4	FINAL	CHANGED IN
Cut	**13**	**0**	E4, section chunks
Missed	**39**	**2**	E4 chunks, E8 filter
Ranking all ranking losses	**86**	**39**	E7 reranker, E8 filter
Misread	**17**	**2**	E7 and E10
Leaks of 520 questions	**51**	**0**	E8, the filter

What is left

39 of the 46 wrong answers are now ranking losses. That is where the next fix belongs.

Finding

Most ranking losses happen before the reranker

The 39 wrong answers lost at ranking, split by where the right passage sat after search.

OUTSIDE THE SHORTLIST

24 of 39

Ranked below 20th by search. The reranker never read them.

RANKED TOO LOW

15 of 39

In the reranker's 20, but it put them below 5th.

The lesson

The reranker can only reorder what search hands it. Most of the remaining problem is search, so a better reranker would fix at most 15 of these.

Worked example

A follow-up that lost its topic

A FOLLOW-UP FROM THE TEST

"What about the Tessa dishwasher?" after "How long should I wait to turn on my Keel washing machine after moving it?"

Search	Sees only "What about the Tessa dishwasher?", not "after moving it". The right passage ranks 11th.	11th
Rerank	The reranker reads it too, but with the same vague question, it moves the passage down to 13th.	13th
Answer	"The passages do not specify how long to wait before turning on the Tessa dishwasher after moving it."	wrong

Traced to: ranked too low

The fix is not the reranker. It is giving search the whole question: rewriting follow-ups first (E6).

Finding

Follow-ups and part codes make up most of the rest

The final system’s 46 wrong answers, by kind of question.



33 of 46

Follow-ups and codes together. E5 (keyword search for codes) and E6 (rewriting follow-ups) each helped these, but neither cleared the rule to join.

So

Tracing points back to two fixes already tested. A narrower rule, applied only to these questions, may be worth another look.

Finding

Some wrong answers are the library's fault

Only 5 wrong answers had the right passage in the prompt. 3 of those 5 are the same problem.

ONE OF THE THREE

How long is the warranty on the Halvard Home Mira robot vacuum from the date of purchase?

A SUPPORT ARTICLE SAYS	THE WARRANTY POLICY SAYS
"a two-year warranty from purchase"	"40 months from delivery"

With both in the prompt, the model reported the conflict and would not choose. Marked wrong, but honest.

The lesson

Tracing turns "the bot is wrong" into "these two documents disagree". The fix is the content, not the model.

S2 E14

Finding

When it should have said “not found”

Of the 120 questions the library cannot answer, the final system answered 13 anyway.

TOPIC COVERED, DETAIL MISSING

9 of 13

The passages are about the right thing, just not the exact point asked.

ABOUT STAFF-ONLY DOCUMENTS

4 of 13

Answered from nearby public passages. No staff text reached the prompt.

The lesson

A nearly relevant passage is the hardest case: close enough to tempt an answer. This is the "refurbished part" problem from E9.

Finding

All three predictions held

The predictions from slide 8, against the result.

Before the reranker	24 of the 39 ranking losses were outside the top 20.	held
Two kinds of question	Follow-ups and codes: 33 of 46, or 72%.	held
Detail, not topic	9 of the 13 were topic covered, detail missing.	held

What this does not prove

- **The judge is taken as given.** In the follow-up example, the judge’s reasoning was wrong, though its verdict was right.
- **First lost is not only cause.** A passage ranked too low might also have been misread.
- **Labels needed.** Tracing needs to know which passage holds each answer. Without labels, a judge must check by hand.
- **Right answers with the passage lost** are not counted as failures.

For engineers

Run it yourself

The whole episode reads three existing runs. No key, no cost.

```
# trace every wrong answer in E2, E4 and the final system
$ uv run python episodes/s2-e14-tracing/analyse.py
```

The function	`first_lost_stage()` in src/lab/rag/score.py.
The examples	One worked example per stage, in results/analysis.json.
Cost	Nothing. No model call.

For leaders

Turn every complaint into a known cause

Here, code traced all 46 wrong answers to a stage, for free. Two in three pointed to the same two problems.

Ask your team

When our assistant gets something wrong, can we say which stage caused it?

Ask your team

Do we log what every stage received and passed on, for every question?

Ask your team

Who fixes it when the cause is our documents disagreeing, not the AI?

Scoreboard

No stage changed. The season, traced.

1 Load	2 **Chunk**	3 Embed	4 Rewrite	5 Search	6 **Filter**	7 **Rerank**	8 **Prompt**	9 Answer	10 **Cite**	11 Score

	WRONG	MISSED	MISREAD	LEAKS
E4 `the like-for-like start`	**157** of 400	**39** of 400	**17** of 400	**51** of 520
Final `after E13`	**46** of 400	**2** of 400	**2** of 400	**0** of 520

> Lime: joined. Coral: tested, not joined. E14 changed nothing; it explains what the season’s changes did.

Decision rule

When an answer is wrong, find the first stage that lost the right passage before changing anything.

Next: S2 E15, When RAG is the wrong answer

RAG finds a passage. What if the answer is not a passage at all?

Where this fits

The series map

S1 LLMs	**S2** Retrieval	**S3** Strategy
S4 Adapting	**S5** Agents	**S6** Economics
S7 Data	**S8** Evaluation	**S9** AI teams
S10 Reliability	**S11** Security	**S12** Delivery
S13 Performance	**S14** Resilience	**S15** Governance

☐ Leadership season

Follow linkedin.com/in/brendan-kelly-irl

Episode guide brendankellyai.github.io

Code github.com/BrendanKellyAI/applied-ai-lab